

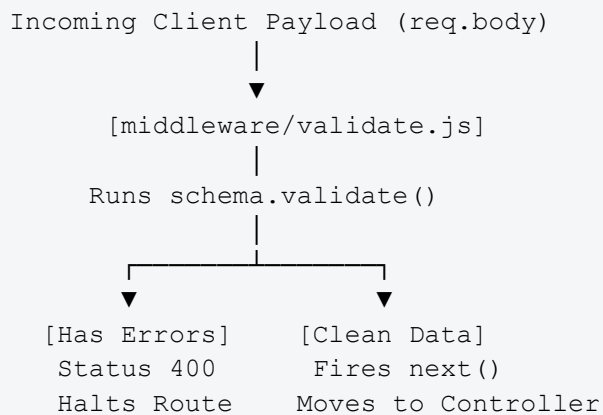
RARITONE UNIFIED BACKEND: SECURITY, VALIDATION & ERROR HANDLING

Engineering Reference & Quality Assurance Blueprint

1. CORE SYSTEM ARCHITECTURE WORKFLOWS

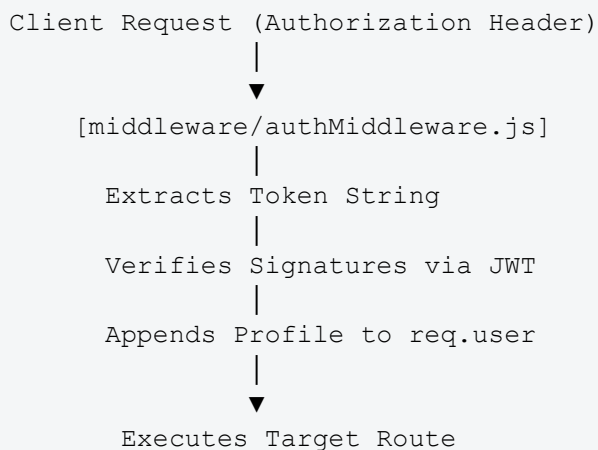
1.1 Request Validation Workflow (Joi Engine Execution)

Every raw payload reaching data manipulation routes is checked immediately. This isolates incoming formatting failures right at the threshold layer, ensuring the controller never handles incomplete or malformed objects.



1.2 Protected Route Workflow (JWT Route Security)

Private endpoints use token parsing sequences to verify authentication claims before resolving client requests.



2. CODE REPOSITORY ARCHITECTURE ANALYSIS

2.1 Centralized Schema Engine (middleware/validate.js)

This component acts as a high-order function intercepting request bodies. By parsing Joi engine schemas and reading error arrays explicitly, it automatically routes error details out with a clean 400 Bad Request code.

2.2 System Gatekeeper Middleware (middleware/authMiddleware.js)

Examines incoming connection context headers for tokens, evaluates cryptographical authenticity via secret keys, pulls verified document records from MongoDB excluding sensitive data tags like passwords, and registers them directly onto the active runtime thread context.

2.3 Error Processing Life cycle Router (middleware/errorMiddleware.js)

This function implements global error catch routing. It catches structural database exceptions (Mongoose structural CastError, duplicate collection keys like Mongo Code 11000, and validation breaks) along with JWT expirations, formatting clean, consistent JSON responses for front-end integration.

3. QUALITY ASSURANCE TESTING MANIFEST (POSTMAN)

Run these baseline quality control test procedures in your workspace environment:

Testing Scenario Target	Postman Setup Setup	Expected Core JSON Signature
Joi Schema Validation Faults	POST /auth/register with a 2-char short password field.	Status: 400 Bad Request { "success": false, "message": "\password" length must be at least 6 characters long" }
Missing Token Protection Access	GET /profile with the Authorization Header left unchecked.	Status: 401 Unauthorized { "success": false, "message": "No token, authorization denied" }

Expired Session Check
Sequence

GET /profile using an old or
manually broken Bearer token
string.

```
Status: 401
Unauthorized
{
  "success": false,
  "message": "Invalid
or expired token"
}
```

1. Request Validation Workflow (validate.js + Joi Schemas)

codebase uses a functional closure factory inside middleware/validate.js.

- **The Process:** When an incoming client request hits a route (like POST /auth/register), validate.js intercepts the request body and runs it against specialized schemas (registerValidator.js or authValidator.js).
- **The Safeguard:** If a user submits an invalid email or a password shorter than 6 characters, Joi isolates the error instantly. The middleware short-circuits the pipeline and returns a standardized 400 Bad Request with the exact message from error.details[0].message. The controller is completely shielded from dirty data.

2. Protected Route Gateway (authMiddleware.js)

This middleware acts as the secure perimeter guard for paths like profile endpoints (GET /profile).

- **Token Extraction:** It reads the incoming Authorization header and handles extracting the token string safely whether the client prefixes it with a standard Bearer string or passes the raw hash string directly.
- **Identity Injection:** After verifying the cryptographical validity using jwt.verify() against your .env file variables, it queries local database using User.findById(decoded.id).select("-password"). Stripping out the password ensures memory context protection before appending the verified record directly onto the request stream (req.user = user) for use downstream.

3. Centralized Error Lifecycle Handling (errorMiddleware.js)

Instead of peppering chaotic try/catch JSON payloads inside every single controller, your system channels all runtime disruptions through an Express global error middleware layer using a four-argument sequence signature: (err, req, res, next).

- **Explicit DB Parsing:** It specifically checks for Mongoose infrastructure problems like a malformed string Object ID (CastError), duplicate data inputs such as

registering an existing email database record (Mongo Error 11000), or Joi schema compilation issues (ValidationError).

- **Session Expiration Isolation:** It cleanly separates active structural failures from identity problems by explicitly isolating native JWT bugs like JsonWebTokenError ("Invalid token") and TokenExpiredError ("Token expired").
- **Environment Protection:** If the system falls back onto a 500 Internal Server Error, it reads process.env.NODE_ENV. If set to "development", it prints out full raw terminal stack traces (err.stack) to help developers fix bugs instantly, while completely hiding those vulnerability leaks from end-users in production.